

IoT Challenge 3: Node-RED and LoRaWAN

Angelica Ortiz 11191225, Daniel Nägele 11180610

April 2026

Contents

1	Node-RED implementation	3
---	-------------------------	---

1 Node-RED implementation

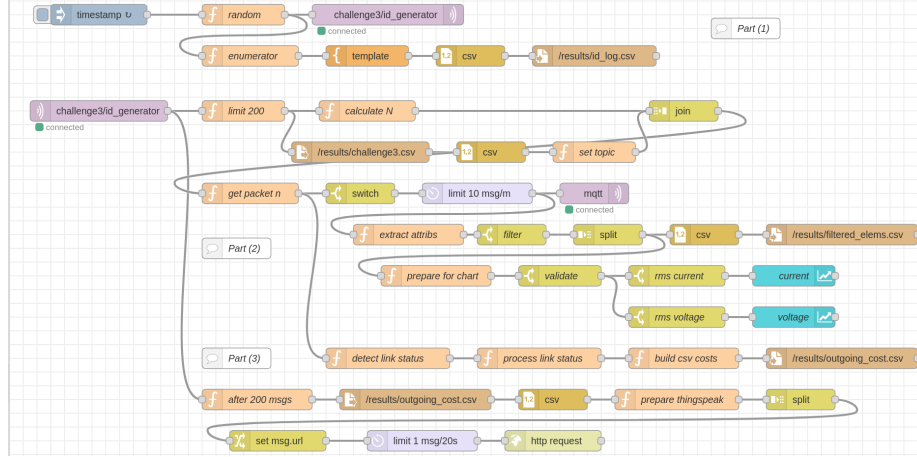


Figure 1: Screenshot of Node-RED Flow

Figure 1 shows a screenshot of the Node-RED flow. This flow implements all the functionality required by the Challenge. Starting in the top left, there is the “timestamp” inject node, which periodically generates a message with the current timestamp. The function node “random” receives this timestamp, converts it from milliseconds to seconds, and generates a random number. The MQTT node then publishes this message. The “enumerator” node then enumerates each incoming message with an incremental row number. Next, the template node simply changes the message field names to match the CSV column names. The output is then converted to CSV and written as a file.

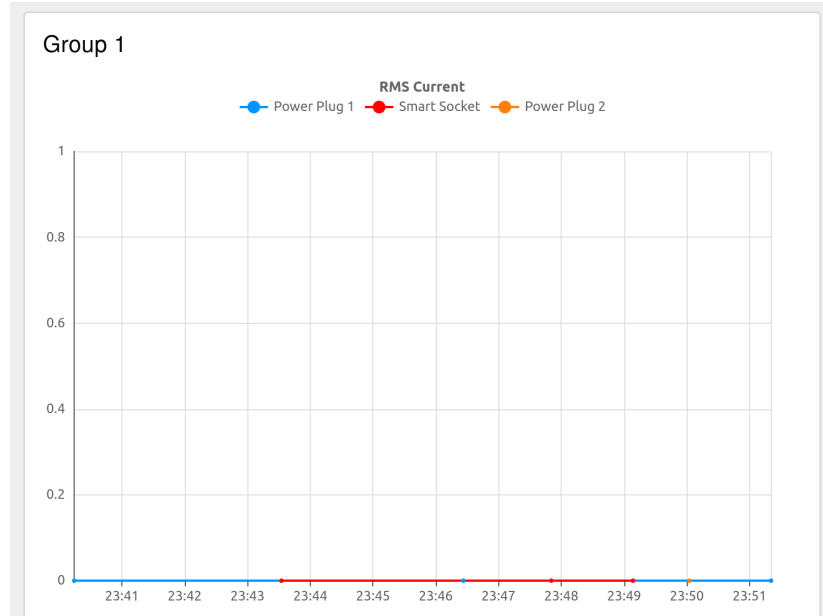


Figure 2: RMS Current Dashboard

The messages sent earlier to the given MQTT topic are received. Based on these messages, the number N (index of the row in the provided CSV file) is calculated and a topic is set for this message. Meanwhile, the CSV file is read and converted and a topic is set for its message using the function node labeled "topic". The resulting messages are then joined together and the topic is used as a key for the respective message. Next, in the function node "get packet n", the n th recorded packet and its data are extracted and validated. In addition, the payload and topic of the message is already prepared as such that it can be sent to the MQTT broker in the specified format. This node provides the basis for the second and third parts of the Challenge. The "filter" only forwards messages that have the ZBEE_ZCL layer. These are then sent to MQTT.

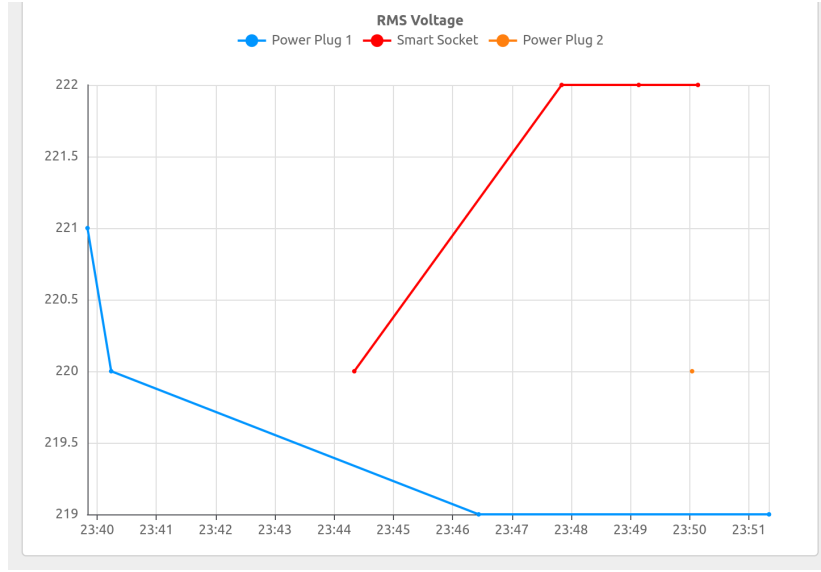


Figure 3: RMS Voltage Dashboard

The provided packet is examined in the “extract attributes” function node as well. The desired attributes are extracted, as well as their data types and values. After that, they are filtered if they have no attributes, and the message is split on a per-attribute basis. These attributes are then written to the corresponding CSV file. Depending on the attribute name, these values are also forwarded to the corresponding chart node. Figures 2 and 3 display screenshots taken from the Node-RED dashboard for the attributes “RMS Current” and “RMS Voltage” respectively.

Additionally, the flow implements the processing of Link Status packets (**Packet Type = Link Status (0x08)**). Immediately after the MQTT subscription node `challenge3/id_generator`, the function node `limit 200` counts all received ID messages. Once exactly 200 messages have been processed, this node stops forwarding additional packets, including ignored packets.

The function node `get packet n` retrieves the packet corresponding to the calculated packet number N from the CSV dataset and stores the complete packet information in `msg.packet` for subsequent processing.

The function node `detect link status` checks the value of the **Packet Type** field contained in `msg.packet`. If the packet type is equal to **Link Status (0x08)**, the message is forwarded to the function node `process link status`. Otherwise, the node returns `null`, and the packet is ignored by this processing branch.

The function node `process link status` extracts the ZigBee source address, the destination addresses contained in the Link Status packet, and their corresponding outgoing costs. Using a regular expression, this information is parsed from the `Command String` field and stored in the flow context variable `outgoing_costs`. If newer Link Status packets are received from the same source, the stored outgoing costs are updated accordingly.

The function node `build csv costs` reads the stored `outgoing_costs` flow context structure and converts it into the required CSV format. Finally, the generated CSV content is stored in `msg.payload` and written to the file `/results/outgoing_cost.csv`.

The function node `prepare thingspeak` receives the parsed rows of `outgoing_cost.csv`. It extracts all source addresses, removes duplicates, and sorts them as hexadecimal values in order to select the smallest source address. Then, it filters the rows to keep only the links associated with this source and sorts their destination addresses in ascending hexadecimal order.

Finally, the node creates one ThingSpeak HTTP request for each destination address. In each request, the corresponding outgoing cost is inserted into `field1`, and the generated URL is stored in `msg.payload.url` for the following nodes.

The `split` node generates one individual HTTP request per destination address. Afterwards, the node `limit 1 msg / 20s` ensures compliance with the ThingSpeak transmission rate by allowing only one message every 20 seconds.

The `change` node `set msg.url` copies the generated URL from `msg.payload.url` into `msg.url`, which is required by the HTTP request node. Finally, the `http request` node transmits the data to the public ThingSpeak channel. The resulting transmission behavior can be observed in Figure 4.

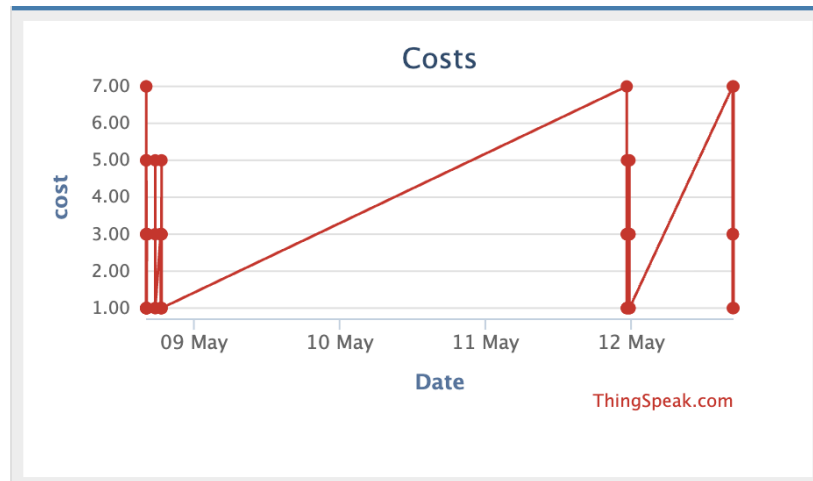


Figure 4: ThingSpeak graph

The channel id of Thingspeak is: 3375015.